East West University

Task 2 Report

Course Code:	CSE 475
Course Name:	Machine Learning
Section:	01
Dataset:	https://ninapro.hevs.ch/instructions/DB2.html
Title:	A Comparative Study on Graph Construction Strategies for Hand Gesture Recognition based on sEMG Data with Graph Neural Networks.
Semester:	Fall 2026

Submitted to

Dr. Raihan Ul Islam

Associate Professor

Department of Computer Science & Engineering

Submitted by

Marzia Hasan	ID: 2024-1-60-213
Marjan Hasan	ID: 2024-1-60-214
Md Kamrul Hasan Rasel	ID:2023-1-60-015
Shohana Khatun	ID:2023-3-50-048

Submission date

12th November, 2026

Table Of Contents	
1.Introduction	03
2.Data and set-up	03
3.Preprocessing and leakage check	03-04
4. Baseline models 4.1 Overfitting check (train vs. test) 4.2 Channels and gestures 4.3 Inference time and model size	04-08
5. Proposed model: EMG-CorrGCN 5.1 (A) Graph construction 5.2 (B) Architecture diagram 5.3 (C) How it works (in plain words	08-10
6. Training and initial experiences 6.1 Learning curves 6.2 Results over 5 seeds 6.3 Comparison with the baselines 6.4 Inference , time and model size	10-12
Discussion	13
Limitations	13
Conclusion	13

Kaggle Link:
https://www.kaggle.com/code/marjan15/group05-ninapro-db2-task2-baseline
https://www.kaggle.com/code/marjan15/group05-ninapro-db2-task2-proposed-model
GitHub Link:
github.com/Marjan15H/Group05_NINAPRODB2_CSE475

1.Introduction :

In this stage we have done two task:
(i) A set of baseline model with leakage-free preprocessing pipeline, and a full evaluation, and (ii) a first Graph Neural Network (GNN) proposal, the construction of its graph and its architectural structure will be compared with the baselines, as will be its initial results.
A written baseline to beat is designated and all numbers in this report are taken from the two notebooks submitted. The difficulty of the task is for a well-founded reason: the model should identify the gestures of new people it has never seen. Given this, with model 1, the signal sent from the muscle varies greatly from subject to subject (electrode position, skin, anatomy), causing the accuracy of each model to be much lower than for a within-subject setting. This is the main theme of our results

2.Data and set-up:

Dataset:

There are 12 sEMG electrodes placed on 40 subjects for 12000 signals recorded at 2000Hz and three exercise tasks (120 .mat files) recorded in NinaPro DB2. There are 207,713,115 samples per channel and 2nd channel. In Task 2 only 17 gestures from Exercise B (1st exercise from DB2) are used.

Windows and features:

Every time, to repeat the gesture, it is one segment. From every active segment we take one random window of 400 samples (200 ms) and compute the RMS of each channel. This yields 1 vector of length 12 for each window. Over all exercises there are 23,640 segments and 11,760 windows (294 per subject); in the case of Exercise B, there will be 4,080 windows, 17 classes.

Subject-independent split:

Subjects are randomly permuted (seed 42) then divided before anything is fitted:

```
Subjects: 40 | windows per subject: min 294, mean 294, max 294
Graph per sample: 12 nodes (one per electrode), 1 feature per node (window RMS)
```

split	subjects	windows	gestures
test	8	2352	49
train	28	8232	49
val	4	1176	49

3. Preprocessing and leakage checks:

The log(RMS) is the preprocessing and StandardScaler fitted against training windows, since this brings the distribution of the EMG amplitude to a good balance, without too much skew to the right. Automatic Checklist executed on notebook, all passed.

	Step	Result	Decision	OK
0	Subject-independent split	28 / 4 / 8 subjects (train / val / test)	no subject appears in two groups	True
1	Missing values	0 non-finite values in the features	no imputation needed	True
2	Scaling	log(RMS) + StandardScaler	scaler was fitted on TRAIN windows only	True
3	Encoding	17 gestures mapped to labels 0..16	LabelEncoder (classes are not ordered, so no o...	True
4	Class imbalance	train windows per class: min 168, max 168	balanced: no resampling or class weights needed	True
5	Feature selection	12 / 12 channels are significant (ANOVA, train...	keep all channels (each channel is a graph node)	True

All checks passed

	F_score	p_value
ch11	62.0071	0.0
ch04	38.6083	0.0
ch09	29.0493	0.0
ch05	28.6039	0.0
ch01	26.1975	0.0
ch03	24.8289	0.0
ch02	23.1222	0.0
ch08	14.8998	0.0
ch10	11.4421	0.0
ch12	11.2773	0.0
ch07	10.9471	0.0
ch06	9.3320	0.0

The ANOVA F-test (train only) ranks the channels as follows: ch11 (F = 62.0), ch04 (38.6), ch09 (29.0), ch05 (28.6), ch01 (26.2), then ch03, ch02, ch08, ch10, ch12, ch07 and ch06 (F = 9.3). No channel is removed – all P-values are < 0.05.

4. Baseline models:

The same features were used in training 7 baselines: Logistic Regression (max_iter 2000), SVM with RBF kernel (C = 10), k-NN (k = 7), Decision Tree (max depth = 12), Random Forest (300 trees), Gradient Boosting (scikit-learn HistGradientBoosting, 200 iterations) and an MLP (hidden layers = 128 and 64, early stopping). All use seed 42. All metric set (accuracy,balanced accuracy,

macro precision, macro recall, macro-F1) and training time are reported. The primary is Macro-F1.

model	val_acc	accuracy	balanced_acc	precision_macro	recall_macro	macro_f1	train_time_s
Random Forest	0.240	0.196	0.196	0.202	0.196	0.182	2.135
SVM (RBF)	0.245	0.189	0.189	0.203	0.189	0.180	0.622
Logistic Regression	0.289	0.194	0.194	0.196	0.194	0.176	0.111
Gradient Boosting	0.255	0.183	0.183	0.188	0.183	0.173	10.699
MLP	0.279	0.184	0.184	0.179	0.184	0.168	1.109
k-NN	0.179	0.159	0.159	0.200	0.159	0.157	0.008
Decision Tree	0.150	0.136	0.136	0.127	0.136	0.126	0.074

```
Baseline to beat: Random Forest (test macro-F1 = 0.182)
```

Our baseline model is Random Forest with the test macro F1 score of 0.182. The top 4 (RF, SVM, LR, Gradient Boosting) models are within 1 point of the other, meaning they are within the noise of an 8 subject test set. All of the models perform about 3 times better than chance (0.059), albeit not anywhere near to usable accuracy.

4.1 Overfitting check (train vs. test):

What the main observation is here is that the performance is far superior on subjects it has been able to learn the content of than on new subjects:

model	train_acc	test_acc	gap
Random Forest	1.000	0.196	0.804
SVM (RBF)	0.707	0.189	0.519
Logistic Regression	0.301	0.194	0.107
Gradient Boosting	1.000	0.183	0.817
MLP	0.411	0.184	0.227
k-NN	0.554	0.159	0.395
Decision Tree	0.647	0.136	0.511

Both algorithms, Random Forest and Gradient Boosting, reach only about 19% on new subjects, while memorising the training subjects, bringing 100% train accuracy. Logistic Regression has the least gap of all (0.107). Thus, the emphasis is not on model flexibility but is on the

subject-to-subject variation that the model should be capable of capturing: if a model is flexible, it is not helpful unless it is robust to subject differences.

4.2 Channels and gestures:

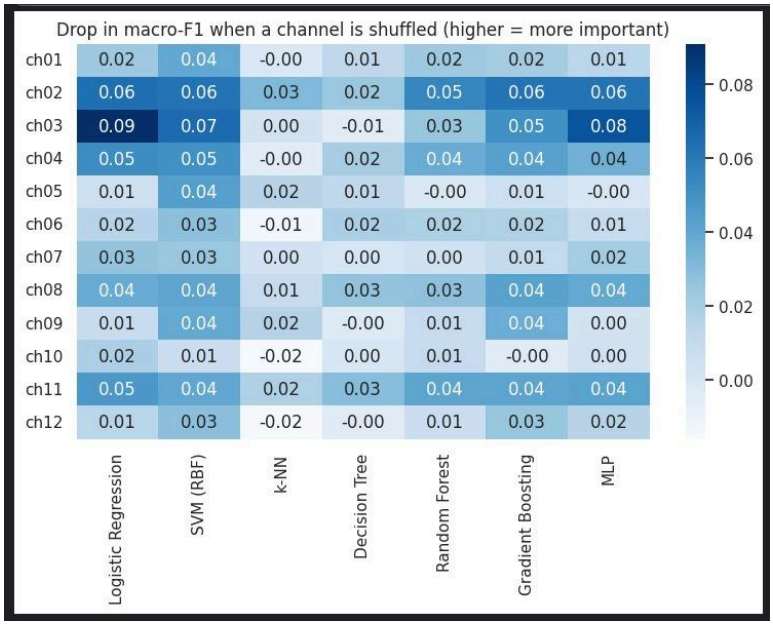


Figure 1. Permutation importance of each channel (drop in macro-F1 when the channel is shuffled), on the validation set, for every baseline

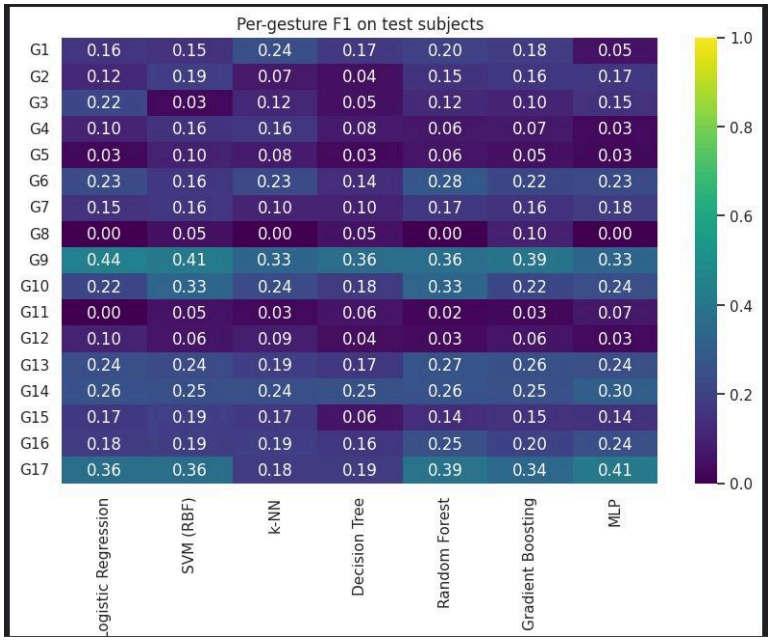


Figure 2. Per-gesture F1 of every baseline on the test subjects

Fifth, from a baseline point, there are a wide variety of difficulties for gestures, as indicated in the classification report. The best F1 values are G17 (0.39), G9 (0.36) and G10 (0.33); the worst

are G8 (0.00), G11 (0.02) and G12 (0.03). Nonsense muscle pattern gestures are confused with similar pattern gestures (Figure 3).

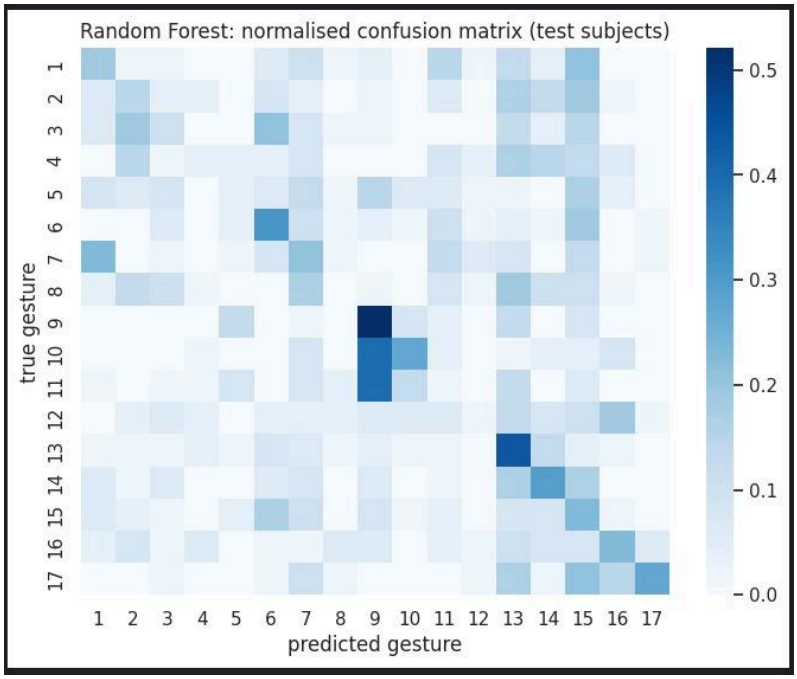


Figure 3. Normalised confusion matrix of the best baseline (Random Forest) on the test subjects.

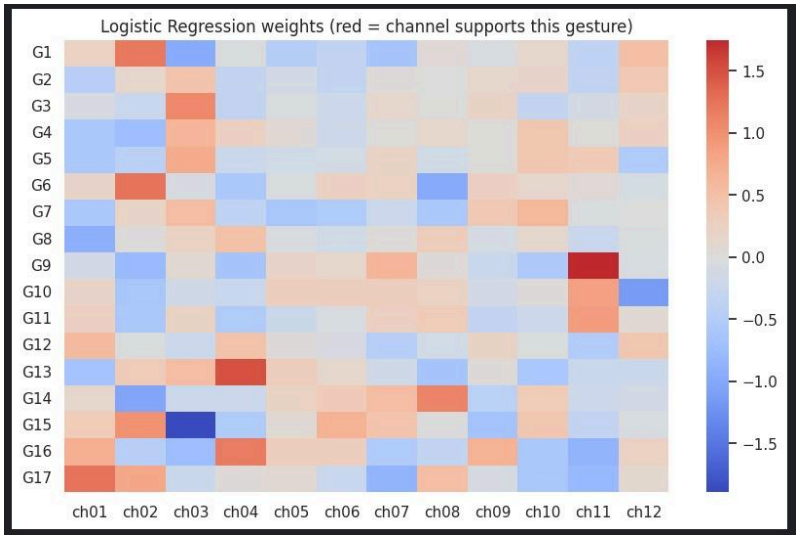


Figure 4. Logistic Regression weights (gesture x channel): which channel supports which gesture

4.3 Inference time and model size:

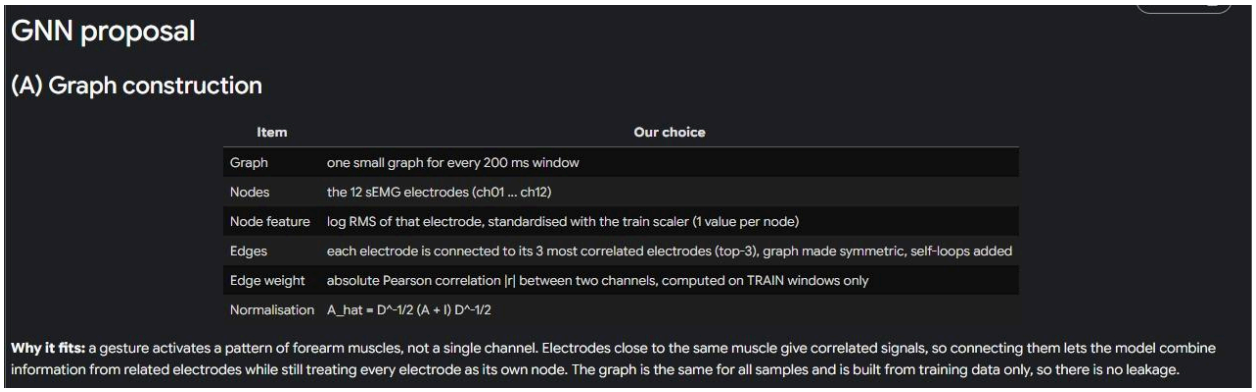
	inference_ms_per_window	model_size_KB
Logistic Regression	0.002	2.538
SVM (RBF)	0.225	599.279
k-NN	0.053	475.361
Decision Tree	0.001	218.248
Random Forest	0.145	123436.495
Gradient Boosting	0.289	11082.070
MLP	0.002	137.788

But the largest model, roughly 120 MB, is the most accurate baseline model (Random Forest).

5. Proposed model: EMG-CorrGCN:

Our proposed network is EMG-CorrGCN (EMG Correlation-graph GCN): a graph convolutional network on a correlation graph of the 12 electrodes.

5.1 (A) Graph construction:



GNN proposal

(A) Graph construction

Item	Our choice
Graph	one small graph for every 200 ms window
Nodes	the 12 sEMG electrodes (ch01 ... ch12)
Node feature	log RMS of that electrode, standardised with the train scaler (1 value per node)
Edges	each electrode is connected to its 3 most correlated electrodes (top-3), graph made symmetric, self-loops added
Edge weight	absolute Pearson correlation |r| between two channels, computed on TRAIN windows only
Normalisation	A_hat = D^-1/2 (A + I) D^-1/2

Why it fits: a gesture activates a pattern of forearm muscles, not a single channel. Electrodes close to the same muscle give correlated signals, so connecting them lets the model combine information from related electrodes while still treating every electrode as its own node. The graph is the same for all samples and is built from training data only, so there is no leakage.

A pattern of forearm muscles is involved in a gesture not really a single channel. When one connects electrodes over the same muscle, the information that comes from these electrodes is related, and a joint analysis of information from such related electrodes can be made in the model without any loss of independence between any two members of the electrode ensemble. The graph is the same for all samples, and is created using only training data – no leakage. This gives a graph of 12 nodes and 26 (non-connecting) edges.

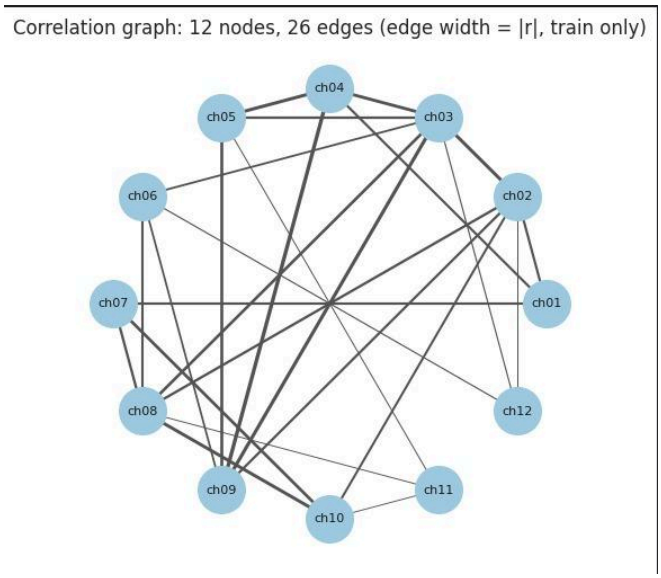


Figure 5. The correlation graph used by EMG-CorrGCN
(edge width = |r|, train data only)

5.2 (B) Architecture diagram:

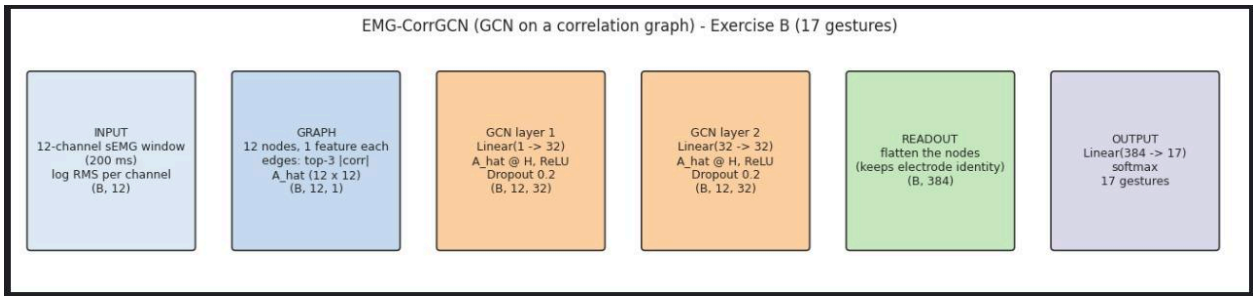


Figure 6. EMG-CorrGCN: input -> graph -> two GCN layers -> flatten readout -> output

he data flow, with tensor shapes (B = batch size): input (B, 12) -> graph node features (B, 12, 1) -> GCN layer 1 Linear(1 -> 32), A_hat @ H, ReLU, Dropout 0.2 -> (B, 12, 32) -> GCN layer 2 Linear(32 -> 32), A_hat @ H, ReLU, Dropout 0.2 -> (B, 12, 32) -> flatten (B, 384) -> Linear(384 -> 17) -> softmax over 17 gestures. The model has 7665 parameters.

5.3 (C) How it works (in plain words):

Every EMG window of 200ms is processed and converted to a small graph of 12 nodes (one for each electrode). Each node only will contain one number, the activity level (log RMS) of the muscle underneath the electrode. The nodes of every GCN layer then transform, with a shared linear layer, their own value and combine it with its neighbors' values, based on their value correlation. Two layers imply that information can flow 2 steps in the graph. While it is possible to perform mean (average) pooling of the nodes in the end, doing so would mask which electrode was being used so we do not do that, but we flatten (show side by side) all 12 node vectors and a

final linear layer assigns a score to each gesture. The model is trained using the cross-entropy method and the best epoch according to the validation accuracy is tested

	setting
Proposed network	EMG-CorrGCN (EMG Correlation-graph GCN)
Task	17-class gesture recognition, Exercise B, subj...
Nodes / node feature	12 electrodes / log RMS (1 value per node)
Graph	top-3 |corr| neighbours, symmetric, self-loops...
GCN layers	2 x (Linear -> A_hat @ H -> ReLU -> Dropout)
Hidden size / dropout	32 / 0.2
Readout	flatten (12 x 32 = 384) -> Linear(384, 17)
Loss / optimiser	cross-entropy / Adam (lr 5e-3, weight decay 1e-4)
Epochs / batch	300 / full batch
Model selection	epoch with the best validation accuracy
Seeds	5 (0 to 4), mean and std reported

6. Training and initial experiences:

6.1 Learning curves:

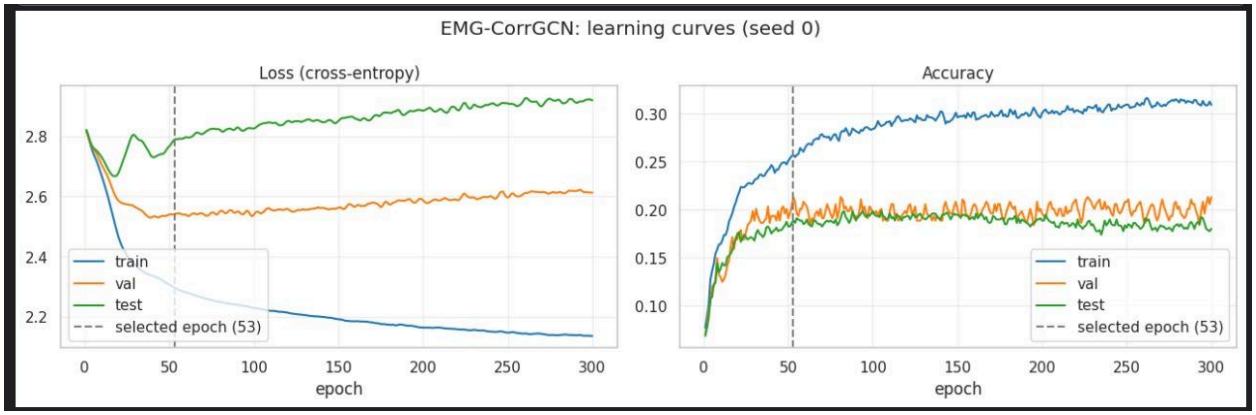


Figure 7. Loss and accuracy of EMG-CorrGCN per epoch (seed 0). The dashed line is the epoch selected on validation accuracy.

	mean	std
train_loss	2.285	0.019
val_loss	2.536	0.007
test_loss	2.789	0.010
train_acc	0.260	0.009
val_acc	0.218	0.003
test_acc	0.187	0.004

While training loss continues to drop, validation error does not get lower after about 50 epochs, and validation and test losses start to increase--we begin to see the network overfitting the training set instead of learning something that will generalize to the other set:The training loss continues to decrease but validation error does not get lower after about 50 epochs; after that, the validation and test losses start to increase; we begin to see that the network is learning what it is seeing, rather than what it is supposed to be learning: The selected epochs over the 5 seeds are 53, 56, 81, 54 and 48. At the selected epoch (mean of 5 seeds), accuracy is 0.260 (train), 0.218 (validation) and 0.187 (test).

6.2 Results over 5 seeds:

```
parameters: 7665
```

seed	val_acc	accuracy	balanced_acc	precision_macro	recall_macro	macro_f1	train_time_s	best_epoch	test_loss
0	0.213	0.189	0.189	0.186	0.189	0.173	10.641	53	2.789
1	0.218	0.190	0.190	0.179	0.190	0.170	10.593	56	2.792
2	0.221	0.190	0.190	0.184	0.190	0.172	10.785	81	2.790
3	0.218	0.185	0.185	0.187	0.185	0.171	10.695	54	2.801
4	0.218	0.180	0.180	0.188	0.180	0.165	10.518	48	2.773

```
EMG-CorrGCN test macro-F1: mean 0.170, std 0.003
```

6.3 Comparison with the baselines:

model	val_acc	accuracy	balanced_acc	precision_macro	recall_macro	macro_f1	train_time_s
Random Forest	0.240	0.196	0.196	0.202	0.196	0.182	2.135
SVM (RBF)	0.245	0.189	0.189	0.203	0.189	0.180	0.622
Logistic Regression	0.289	0.194	0.194	0.196	0.194	0.176	0.111
Gradient Boosting	0.255	0.183	0.183	0.188	0.183	0.173	10.699
EMG-CorrGCN (mean of 5 seeds)	0.218	0.187	0.187	0.185	0.187	0.170	10.647
MLP	0.279	0.184	0.184	0.179	0.184	0.168	1.109
k-NN	0.179	0.159	0.159	0.200	0.159	0.157	0.008
Decision Tree	0.150	0.136	0.136	0.127	0.136	0.126	0.074

```
Baseline to beat: Random Forest (test macro-F1 = 0.182)
EMG-CorrGCN: 0.170 +- 0.003
Verdict: EMG-CorrGCN is worse than the best baseline.
Random guess = 0.059. The test set has only 8 subjects, so gaps of 1-2 points are not reliable.
```

Verdict: EMG-CorrGCN (0.170 ± 0.003) is almost as bad as the best baseline (Random Forest, 0.182) and ranks 5th of 8 models, a similar performance to the Gradient Boosting model (0.173), and the MLP (0.168). std value for seed-to-seed is small (0.003) but the gap to Random Forest does not vary in significance across different seeds, due to the limited size of test set (8 test

subjects, 1-2 points should not be interpreted too much). A well-designed significance test over 5-fold cross-validation is set to be carried out in Task 3.

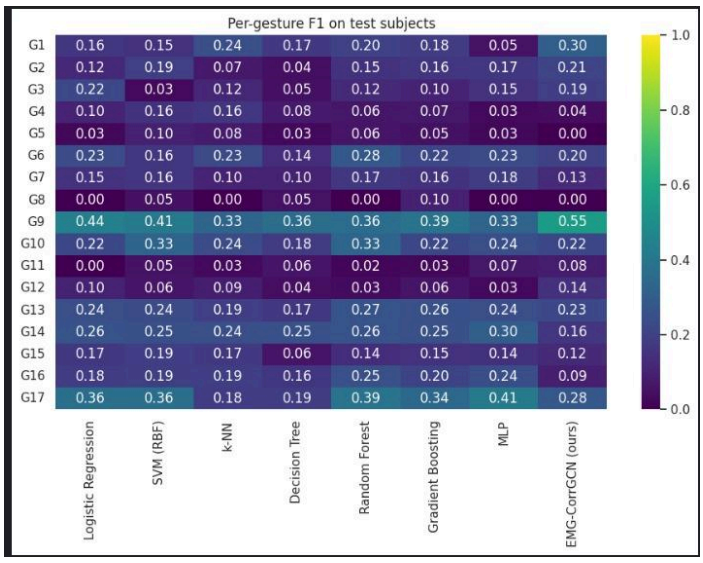


Figure 8. Per-gesture F1 on the test subjects for all baselines and EMG-CorrGCN (seed 0)

6.4 Inference , time and model size:

	inference_ms_per_window	model_size_KB
Logistic Regression	0.002	2.538
SVM (RBF)	0.225	599.279
k-NN	0.053	475.361
Decision Tree	0.001	218.248
Random Forest	0.145	123436.495
Gradient Boosting	0.289	11082.070
MLP	0.002	137.788
EMG-CorrGCN (ours)	0.003	29.941

Model size: pickle size for the sklearn models, parameters x 4 bytes for EMG-CorrGCN.

Very light: 7,665 parameter (only 30KB), which is the computed number of parameters x 4 bytes; and about 0.003 ms per window, which is about 4,000 times smaller than Random Forest. A little slower than the majority of baselines (10.6 s / 300 full-batch epochs / 300 ops steps) but acceptable training time. Note that the size of the models is a pickle size or number of model parameters, with the GCN model having only approximate comparison.

7. Discussion:

The question that still remains is, why doesn't EMG-CorrGCN beat the best baseline yet. There are three reasons all of which are visible to us in our results:

• Very small node feature. Only a single number (log RMS) at each node. The graph provides only a handful of additional scalar values to what the graph model senses with such a lightweight feature.

• Weak graph. The channels are not strongly correlated with each other (mean r = 0.16), so the top-3 edges, which have the highest relevance in this graph, are not informative; the exact muscles layout cannot be expressed as a graph.

• Subject variation dominates. For all models, including the GCN, all the parameters were overfitted on the 28 subjects used for training (by test accuracy 0.19 for the GCN). Compared to all other models tested, the GCN has the smallest train-test gap (0.073), which indicates that the graph helps to work as a regulariser, but does not immediately reflect in increased test accuracy.

8.Limitations.:

(i)The training set is relatively small, only one random window is used per repetition (2,856 windows).

(ii) A fixed split of 8 test subjects yields noisily estimated values.

(iii) We're also missing XGBoost/LightGBM, a 1D-CNN, and some other less widely used libraries; LR, SVM, k-NN, Decision Tree, Random Forest, Gradient Boosting (HistGradientBoosting) and MLP are part of the baseline.

(iv) No tuning of hyper-parameters of the baselines.

9. Conclusion:

For Exercise B, we created a pipeline that is both leakage free and independent of the subject and trained 7 baselines. The baseline to compare: Random Forest (macro-F1 = 0.182). The biggest challenge is learning to transfer their models to new topics, rather than the capacity of the models themselves. Our model, called EMG-CorrGCN, achieves 0.170 ± 0.003, which is slightly smaller, but close to the best one in the baseline methods, Gradient Boosting and MLP, both are about 4,000 times smaller than the best method, and their overfitting gap is smaller.